

(<https://github.com/spacetelescope/asdf-standard>)

 Fork me on GitHub

YAML Schema

YAML Schema is a small extension to JSON Schema Draft 4 (<http://json-schema.org/draft-04/json-schema-validation.html>) that adds some features specific to YAML. Understanding JSON Schema (<http://spacetelescope.github.io/understanding-json-schema/>) provides a good resource for understanding how to use JSON Schema, and further resources are available at json-schema.org (<http://json-schema.org>). A working understanding of JSON Schema is assumed for this section, which only describes what makes YAML Schema different from JSON Schema.

Writing a new schema is described in [Designing a new tag and schema](#) ([../schemas.html#extending-asdf](#)).

Note

The YAML Schema currently does not require either the `id` or `tag` keywords. The `id` keyword is not included in the YAML Schema since it is actually inherited from the base JSON Schema standard. However, it may become mandatory in a future version of the YAML Standard. The `tag` keyword may also eventually become mandatory, although the motivation for this is somewhat weaker.

YAML Schema

Description

A metaschema extending JSON Schema's metaschema to add support for some YAML-specific constructions.

Outline

Schema Definitions

Internal Definitions

Original Schema

Schema Definitions

This node must validate against **all** of the following:

- <http://json-schema.org/draft-04/schema> (<http://json-schema.org/draft-04/json-schema-validation.html>)
- This type is an object with the following properties:

tag

string

A fully-qualified YAML tag name that should be associated with the object type returned by the YAML parser; for example, the object must be an instance of the class registered with the parser to create instances of objects with this tag. Implementation of this validator is optional and depends on details of the YAML parser.

Minimum length: 6

propertyOrder

array

 1.0.2 ▼

Specifies the default order of the properties when writing out. Any keys not listed in **propertyOrder** will be in arbitrary order at the end. This field applies only to nodes with **object** type.

No length restriction

Items in the array are restricted to the following types:

string

No length restriction

flowStyle

string

Specifies the default serialization style to use for an array or object. YAML supports multiple styles for arrays/sequences and objects/maps, called “block style” and “flow style”. For example:

```
Block style: !!map
  Clark : Evans
  Ingy  : döt Net
  Oren   : Ben-Kiki

Flow style: !!map { Clark: Evans, Ingy: döt Net, Oren: Ben-Kiki }
```

This property gives a hint to the tool outputting the YAML which style to use. If not provided, the library is free to use whatever heuristics it wishes to determine the output style. This property does not enforce any particular style on YAML being parsed.

No length restriction

Only the following values are valid for this node:

- **block**
- **flow**

style

string

Specifies the default serialization style to use for a string. YAML supports multiple styles for strings:

```
Inline style: "First line\nSecond line"

Literal style: |
  First line
  Second line

Folded style: >
  First
  line

  Second
  line
```

This property gives a hint to the tool outputting the YAML which style to use. If not provided, the library is free to use whatever heuristics it wishes to determine the output style. This property does not enforce any particular style on YAML being parsed.

 1.0.2 ▼

No length restriction

Only the following values are valid for this node:

- inline
- literal
- folded

examples

array

A list of examples to help document the schema. Each pair is a prose description followed by a string containing YAML content. For example:

```
examples:
-
  - Complex number: 1 real, -1 imaginary
  - "!complex 1-1j"
  type: array
  items:
```

No length restriction

Items in the array are restricted to the following types:

array

No length restriction

The first 2 items in the list must be the following types:

string

No length restriction

This node must validate against **any** of the following:

- **string**
No length restriction
- **object**

additionalItems

object

Details

items

object

Details

additionalProperties

object

Details

definitions

object

object

properties

object

object**patternProperties**

object

object**dependencies**

object

object**allOf**

schemaArray

anyOf

schemaArray

oneOf

schemaArray

not

()

Internal Definitions

schemaArray

array

Minimum length: 1

Items in the array are restricted to the following types:

()

Original Schema

```
%YAML 1.1
---
$schema: "http://json-schema.org/draft-04/schema"
id: "http://stsci.edu/schemas/yaml-schema/draft-01"
title:
  YAML Schema
description: |
  A metaschema extending JSON Schema's metaschema to add support for
  some YAML-specific constructions.
allOf:
- $ref: "http://json-schema.org/draft-04/schema"
- type: object
  properties:
    tag:
      description: |
        A fully-qualified YAML tag name that should be associated
        with the object type returned by the YAML parser; for
        example, the object must be an instance of the class
        registered with the parser to create instances of objects
        with this tag. Implementation of this validator is optional
        and depends on details of the YAML parser.
      type: string
      minLength: 6

    propertyOrder:
      description: |
        Specifies the default order of the properties when writing
        out. Any keys not listed in **propertyOrder** will be in
        arbitrary order at the end. This field applies only to nodes with
        **object** type.
      type: array
      items:
        type: string

    flowStyle:
      description: |
        Specifies the default serialization style to use for an
        array or object.  YAML supports multiple styles for
        arrays/sequences and objects/maps, called "block style" and
        "flow style".  For example::

        Block style: !!map
          Clark : Evans
          Ingy  : döt Net
          Oren  : Ben-Kiki

        Flow style: !!map { Clark: Evans, Ingy: döt Net, Oren: Ben-Kiki }

      This property gives a hint to the tool outputting the YAML
      which style to use.  If not provided, the library is free to
      use whatever heuristics it wishes to determine the output
      style.  This property does not enforce any particular style
      on YAML being parsed.
```

```

type: string
enum: [block, flow]

```

style:

```

description: |
  Specifies the default serialization style to use for a string.
  YAML supports multiple styles for strings:

```

```

```yaml
 Inline style: "First line\nSecond line"

 Literal style: |
 First line
 Second line

 Folded style: >
 First
 line

 Second
 line
```

```

This property gives a hint to the tool outputting the YAML which style to use. If not provided, the library is free to use whatever heuristics it wishes to determine the output style. This property does not enforce any particular style on YAML being parsed.

```

type: string
enum: [inline, literal, folded]

```

examples:

```

description: |
  A list of examples to help document the schema. Each pair
  is a prose description followed by a string containing YAML
  content. For example:

```

```

```yaml
 examples:
 -
 - Complex number: 1 real, -1 imaginary
 - "!complex 1-1j"
 type: array
 items:

```

```

type: array
items:
 type: array
 items:
 - type: string
 - anyOf:
 - type: string
 - type: object

```

*# Redefine JSON schema validators in terms of this document so that*

```

we can check nested objects:
additionalItems:
 anyOf:
 - type: boolean
 - $ref: "#"
items:
 anyOf:
 - $ref: "#"
 - $ref: "#/definitions/schemaArray"
additionalProperties:
 anyOf:
 - type: boolean
 - $ref: "#"
definitions:
 type: object
 additionalProperties:
 $ref: "#"
properties:
 type: object
 additionalProperties:
 $ref: "#"
patternProperties:
 type: object
 additionalProperties:
 $ref: "#"
dependencies:
 type: object
 additionalProperties:
 anyOf:
 - $ref: "#"
 - $ref: "http://json-schema.org/draft-04/schema#definitions/stringArray"
allOf:
 $ref: "#/definitions/schemaArray"
anyOf:
 $ref: "#/definitions/schemaArray"
oneOf:
 $ref: "#/definitions/schemaArray"
not:
 $ref: "#"

definitions:
 schemaArray:
 type: array
 minItems: 1
 items:
 $ref: "#"
...

```